PYTHON CHEAT SHEET · 05 OF 05

Operator Overloading & bytearray

Teach your own classes to use +, ==, [] and len() through magic methods, then work with raw, mutable binary data.

MECHANISM	GOLDEN RULE	BYTES	BYTEARRAY
Dunder (magic) methods	**Return NotImplemented, don't raise**	**Immutable, items 0-255**	**Mutable bytes, edit in place**

01 How operator overloading works

When Python evaluates `a + b` it calls `a.__add__(b)`. If that method is missing or returns `NotImplemented`, Python tries the **reflected** method `b.__radd__(a)`. Define the special method and your objects work with the operator. `a += b` uses `__iadd__` when it exists and falls back to `a = a + b` otherwise.

Operator	Method	Reflected (right side)	In-place
`+`	`__add__`	`__radd__`	`__iadd__`
`-`	`__sub__`	`__rsub__`	`__isub__`
`*`	`__mul__`	`__rmul__`	`__imul__`
`/`	`__truediv__`	`__rtruediv__`	`__itruediv__`
`//`	`__floordiv__`	`__rfloordiv__`	`__ifloordiv__`
`%`	`__mod__`	`__rmod__`	`__imod__`
`**`	`__pow__`	`__rpow__`	`__ipow__`
`@`	`__matmul__`	`__rmatmul__`	`__imatmul__`
`& | ^`	`__and__ __or__ __xor__`	`__rand__ __ror__ __rxor__`	`__iand__ __ior__ __ixor__`
`<< >>`	`__lshift__ __rshift__`	`__rlshift__ __rrshift__`	`__ilshift__ __irshift__`

You write	Python calls	You write	Python calls
`-x +x ~x`	`__neg__ __pos__ __invert__`	`len(x)`	`__len__`
`abs(x)`	`__abs__`	`x[i] x[i] = v del x[i]`	`__getitem__ __setitem__ __delitem__`
`x == y x != y`	`__eq__ __ne__`	`v in x`	`__contains__`
`x < y x <= y`	`__lt__ __le__`	`for v in x`	`__iter__ (and __next__)`
`x > y x >= y`	`__gt__ __ge__`	`x(...)`	`__call__`
`str(x) repr(x)`	`__str__ __repr__`	`hash(x)`	`__hash__`
`bool(x) if x:`	`__bool__ (else __len__)`	`with x:`	`__enter__ __exit__`

02 Example: a Vector class

vector.py

```python
class Vector:
    """A 2-D vector that works with the usual operators."""

    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):                       # what the REPL and repr() show
        return f"Vector({self.x}, {self.y})"

    def __add__(self, other):                 # v + w
        if not isinstance(other, Vector):
            return NotImplemented             # let Python try the other side / raise TypeError
        return Vector(self.x + other.x, self.y + other.y)

    def __sub__(self, other):                 # v - w
        if not isinstance(other, Vector):
            return NotImplemented
        return Vector(self.x - other.x, self.y - other.y)

    def __mul__(self, k):                     # v * 3  (scalar multiplication)
        if not isinstance(k, (int, float)):
            return NotImplemented
        return Vector(self.x * k, self.y * k)

    __rmul__ = __mul__                        # 3 * v  (reflected version)

    def __neg__(self):                        # -v
        return Vector(-self.x, -self.y)

    def __abs__(self):                        # abs(v) = length
        return (self.x ** 2 + self.y ** 2) ** 0.5

    def __eq__(self, other):                  # v == w
        if not isinstance(other, Vector):
            return NotImplemented
        return (self.x, self.y) == (other.x, other.y)

    def __hash__(self):                       # required again once __eq__ is defined
        return hash((self.x, self.y))

    def __bool__(self):                       # the zero vector is falsy
        return bool(self.x or self.y)

    def __getitem__(self, i):                 # v[0], v[1], and unpacking
        return (self.x, self.y)[i]
```

```
>>> v = Vector(3, 4)
>>> w = Vector(1, 2)
>>> v + w
Vector(4, 6)
>>> v - w
Vector(2, 2)
>>> v * 2
Vector(6, 8)
>>> 2 * v                                     # falls back to __rmul__
Vector(6, 8)
>>> -v
Vector(-3, -4)
>>> abs(v)
5.0
```

```
>>> v == Vector(3, 4)
True
>>> v != w                                          # __ne__ is derived from __eq__
True
>>> bool(Vector(0, 0))
False
>>> x, y = v                                        # unpacking works through __getitem__
>>> x, y
(3, 4)
>>> {v: "a point"}[Vector(3, 4)]                    # hashable, so it can be a dict key
'a point'
>>> v + 5
TypeError: unsupported operand type(s) for +: 'Vector' and 'int'
>>> v += w                                          # no __iadd__: becomes v = v + w
>>> v
Vector(4, 6)
```

03 Ordering with @total_ordering

Write `__eq__` and **one** ordering method; `functools.total_ordering` fills in the other three.

version.py

```python
from functools import total_ordering

@total_ordering
class Version:
    def __init__(self, major, minor):
        self.major, self.minor = major, minor

    def __eq__(self, other):
        return (self.major, self.minor) == (other.major, other.minor)

    def __lt__(self, other):
        return (self.major, self.minor) < (other.major, other.minor)

    def __repr__(self):
        return f"v{self.major}.{self.minor}"
```

```
>>> Version(3, 12) < Version(3, 13)
True
>>> Version(3, 13) >= Version(3, 13)                # generated by total_ordering
True
>>> sorted([Version(3, 13), Version(2, 7), Version(3, 9)])
[v2.7, v3.9, v3.13]
>>> max(Version(3, 13), Version(3, 9))
v3.13
```

04 Rules of thumb

- Binary operators (`+`, `-`, `*` …) should **return a new object**. Only the in-place versions (`__iadd__` …) change `self`, and they must `return self`.
- For an unsupported operand type, `return NotImplemented` instead of raising. Python then tries the reflected method and finally raises `TypeError` itself.
- Always give a class a `__repr__`. Add `__str__` only when you want a friendlier display.
- Defining `__eq__` without `__hash__` makes instances unhashable. Define both, or leave it unhashable on purpose.
- To let `sum(vectors)` work, add `__radd__` that accepts `0`, because `sum` starts from `0 + first`.
- You cannot overload `and`, `or`, `not`, `is`, assignment `=` or attribute access `.`, and you cannot invent new operators. Keep each operator's meaning natural.

05 bytes vs bytearray

	bytes	bytearray
Mutable	No	Yes: items, slices, append, extend, insert, pop, remove
Literal	b"abc"	None: `bytearray(b"abc")`
Hashable (dict key)	Yes	No
Items	Integers 0 to 255	Integers 0 to 255
Slice returns	bytes	bytearray
Typical use	Constants, keys, network and file data you only read	Buffers you build, edit or fill in place

```
>>> bytes(3)                                    # three zero bytes
b'\x00\x00\x00'
>>> bytearray(3)
bytearray(b'\x00\x00\x00')
>>> bytearray(b"abc")
bytearray(b'abc')
>>> bytearray([72, 101, 108, 108, 111])         # from a list of ints
bytearray(b'Hello')
>>> bytearray("héllo", "utf-8")                 # text needs an encoding
bytearray(b'h\xc3\xa9llo')
>>> "héllo".encode("utf-8")                     # str -> bytes
b'h\xc3\xa9llo'
>>> b"h\xc3\xa9llo".decode("utf-8")            # bytes -> str
'héllo'
>>> bytearray.fromhex("48 65 6c 6c 6f")
bytearray(b'Hello')
```

06 Indexing, slicing and editing in place

```
>>> ba = bytearray(b"hello")
>>> ba[0], ba[1:3]                              # an item is an INT, a slice is a bytearray
(104, bytearray(b'el'))
>>> ba[0] = 72                                  # assign an int from 0 to 255
>>> ba
bytearray(b'Hello')
>>> ba[1:3] = b"EY"                             # slice assignment may change the length
>>> ba.append(33)                               # append ONE int
>>> ba.extend(b"!?")                            # extend with bytes-like data
>>> ba.insert(0, 62)
>>> ba                                          # inspect the buffer so far
bytearray(b'>HEYlo!!?')
>>> ba.pop()                                    # remove and return the last int
63
>>> ba.remove(33)                               # remove the first item equal to 33
>>> ba
bytearray(b'>HEYlo!')
>>> ba[0] = 256
ValueError: byte must be in range(0, 256)
>>> ba[0] = "H"
TypeError: 'str' object cannot be interpreted as an integer
```

07 Methods, conversion and zero-copy views

```
>>> data = bytearray(b"  Hello, Bytes  ")
>>> data.strip().upper()                                # str-like methods return new bytearrays
bytearray(b'HELLO, BYTES')
>>> data.split(b",")
[bytearray(b'  Hello'), bytearray(b' Bytes  ')]
>>> data.replace(b"Bytes", b"World")
bytearray(b'  Hello, World  ')
>>> data.find(b"Hello")
2
>>> data.hex(" ")                                       # human-readable hex dump
'20 20 48 65 6c 6c 6f 2c 20 42 79 74 65 73 20 20'
>>> bytes(data)                                         # freeze into immutable bytes
b'  Hello, Bytes  '
>>> data.reverse()                                      # in place
>>> data.decode()
'  setyB ,olleH  '
>>> (1025).to_bytes(2, "big"), (1025).to_bytes(2, "little")
(b'\x04\x01', b'\x01\x04')
>>> int.from_bytes(b"\x04\x01", "big")
1025
>>> import struct
>>> struct.pack(">HB", 1025, 7)                         # big-endian uint16 + uint8
b'\x04\x01\x07'
>>> struct.unpack(">HB", b"\x04\x01\x07")
(1025, 7)
>>> buf = bytearray(b"abcdef")
>>> view = memoryview(buf)
>>> view[0:3] = b"XYZ"                                  # writes straight into buf, no copy
>>> buf
bytearray(b'XYZdef')
>>> import io
>>> chunk = bytearray(4)
>>> io.BytesIO(b"0123456789").readinto(chunk)           # fill an existing buffer, no new allocation
4
>>> chunk
bytearray(b'0123')
```

Gotchas

```
>>> bytearray(5), bytearray([5])                        # 5 zero bytes vs ONE byte of value 5
(bytearray(b'\x00\x00\x00\x00\x00'), bytearray(b'\x05'))
>>> b"A"[0], b"A"[0:1]                                  # int vs one-byte bytes
(65, b'A')
>>> b"a" + "b"
TypeError: can't concat str to bytes
```

08 Putting it together: operator overloading on a bytearray

`ByteBuffer` wraps a `bytearray` and teaches Python to use `+`, `+=`, `^`, `==`, `len()`, indexing, slicing and `bytes()` on it.

byte_buffer.py

```
class ByteBuffer:
    """A tiny mutable buffer built on bytearray."""

    def __init__(self, data=b""):
        self._data = bytearray(data)

    def __repr__(self):                         # ByteBuffer(02 05 ...)
        return f"ByteBuffer({self._data.hex(' ')})"

    def __len__(self):                          # len(buf)
        return len(self._data)

    def __getitem__(self, i):                   # buf[0]  or  buf[2:7]
        part = self._data[i]
        return part if isinstance(i, int) else ByteBuffer(part)

    def __setitem__(self, i, value):            # buf[-1] = 0x74
        self._data[i] = value

    def __add__(self, other):                   # buf + buf -> new buffer
        if not isinstance(other, ByteBuffer):
            return NotImplemented
        return ByteBuffer(self._data + other._data)

    def __iadd__(self, other):                  # buf += b"..." -> in place
        self._data.extend(other._data if isinstance(other, ByteBuffer) else other)
        return self

    def __xor__(self, key):                     # buf ^ 0x5A -> XOR every byte
        return ByteBuffer(b ^ key for b in self._data)

    def __eq__(self, other):                    # buf == buf
        return isinstance(other, ByteBuffer) and self._data == other._data

    __hash__ = None                             # mutable, so deliberately unhashable

    def __bytes__(self):                        # bytes(buf)
        return bytes(self._data)


packet = ByteBuffer(b"\x02\x05") + ByteBuffer(b"HELLO")     # header + payload
print("packet     :", packet)
print("length     :", len(packet))
print("first byte :", packet[0])
print("payload    :", bytes(packet[2:7]))

packet += b"\x00"                                           # reserve a checksum byte
packet[-1] = sum(bytes(packet[2:7])) % 256                  # fill it in
print("with csum  :", packet)

scrambled = packet ^ 0x5A                                   # XOR "encrypt"
print("scrambled  :", scrambled)
print("round trip :", (scrambled ^ 0x5A) == packet)
```

Output

```
packet     : ByteBuffer(02 05 48 45 4c 4c 4f)
length     : 7
first byte : 2
payload    : b'HELLO'
with csum  : ByteBuffer(02 05 48 45 4c 4c 4f 74)
scrambled  : ByteBuffer(58 5f 12 1f 16 16 15 2e)
round trip : True
```